Programme de colle - Semaine 8 (du 26 octobre)

La colle est constituée d'une démonstration ou exercice de cours (parmi les énoncés ci-dessous marqués par une (*)) (20min max) suivi d'un ou plusieurs exercices portant sur le programme ci-dessous.

1 Langages formels. Langages réguliers

- Mots : alphabet, mot sur un alphabet, concaténation, puissance d'un mot, structure de monoïde $(\Sigma^*, .)$.
- Préfixes, suffixes, facteurs, sous-mots.
- Langages : opérations ensemblistes sur les langages (union, intersection, différence, complémentaire), concaténation de deux langages, étoile de Kleene d'un langage,
- Langages réguliers (aussi appelés langages rationnels): définition inductive de la classe des langages réguliers, **tout langage fini est régulier (*)**
- Expressions régulières : définition inductive, langage dénoté par une expression régulière (notation $\mu(e)$)
- **Un langage est régulier si et seulement si il existe une expression régulière qui le dénote (*)** [NB : interroger sur un seul sens uniquement, chaque sens se démontre par induction]
- Expressions régulières équivalentes
- Expressions régulières étendues : par exemple $e?$, $e+$, $e\{3, 5\}$, etc. Aucune syntaxe n'est exigible mais il faut savoir comprendre une expression régulière étendue et la traduire en expression régulière du cours équivalente (ex : $e? \equiv (e|\varepsilon)$).

2 Parcours de graphes

Les élèves doivent pouvoir lire, écrire ou modifier un algorithme de parcours de graphe en pseudo-code, en C ou en OCaml

- Définitions usuelles : graphe orienté, non orienté, sommet/nœud, arête/arc, voisins (entrants/sortants), degré, chemins, cycles, graphe pondéré, etc.
- Représentation par matrice d'adjacence ou listes d'adjacence.
- Distance entre deux sommets x et y, définie comme le poids d'un chemin de poids de minimal, notée $\delta(x, y)$

- Parcours de graphes (voir le document : "tous les algos" sur la page du cours)
  - Parcours générique (Algo 1)
  - Parcours en profondeur : avec une pile (Algo 2) ou par récursivité (Algo 3)
  - Parcours en largeur : avec une file (Algo 4) et en version simplifiée (Algo 5)
  - Parcours précédents avec construction de l'arborescence du parcours (Algos 6, 7, 8)

- **Algorithme de Dijkstra**
  - Algorithme fonctionnant sur des graphes pondérés avec des **poids positifs** implémenté à l'aide d'une file de priorité (Algo 9)
  - **L'algorithme de Dijkstra garantit l'invariant suivant (*) :**

$$(I) \qquad \forall x \in S, \quad x \text{ est fermé} \Rightarrow g[x] = \delta(x_d, x)$$

    avec $g[x]$ la valeur affectée à x par l'algorithme de Dijkstra et x_d le sommet de départ choisi. En particulier cela justifie que l'algorithme de Dijkstra construit une arborescence de chemins optimaux (pour les sommets fermés).

- **Algorithme A* : recherche informée**
  - L'algorithme A* a pour objectif de trouver un chemin optimal entre un sommet de départ x_d et un sommet cible x_t.
  - L'algorithme A* utilise une *fonction heuristique* $h : S \to \mathbb{R}^+$ telle que $h(x)$ est une estimation de la distance à de x à x_t. Exemples vus en cours : distance euclidienne (à vol d'oiseau) lorsque le graphe est inclus dans le plan euclidien (ex : carte routière) et distance de Manhattan (villes quadrillées, grilles, etc). L'heuristique vérifie $h(x_t) = 0$.
  - Algorithme et illustration présentés en cours (Algo 10) : implémenté à l'aide d'une file de priorité : je demande aux élèves pour l'intant de savoir exécuter l'algorithme à la main sur un petit graphe et de bien comprendre les différences avec l'algorithme de Dijkstra (choix du sommet ouvert qui minimise $f = g + h$, réouverture possible des sommets fermés, arrêt lorsqu'on ferme x_t).
  - Heuristiques admissible et monotone
    - ⋆ Définitions
    - ⋆ **Si une heuristique est monotone alors elle est admissible (*)**
    - ⋆ Si l'heuristique est admissible l'algorithme A* construit un chemin optimal.
    - ⋆ Si l'heuristique est de plus monotone, il est n'est pas nécessaire de réouvrir les sommets fermés.

3 Automates finis

- **Automates finis déterministes (afd)**
  - Définis comme un quadruplet (Q, q_0, F, δ) avec Q ensemble des états, q_0 état initial, F états finaux, $\delta : Q \times \Sigma \to Q$ fonction (partielle) de transition.
  - Définition, notion de calcul dans un afd, notion de blocage.
  - Langage reconnu (ou accepté) par automate : calcul réussi, mots acceptés. Classe des langages reconnaissables notée $\mathrm{REC}(\Sigma^*)$.
  - Accessibilité et co-accessibilité; afd émondé
  - Afd complet; complétion d'un afd
  - **Automate complémentaire : Si L est reconnaissable alors $\bar{L}$ l'est (*)**
  - Automate produit. **Si A_1 et A_2 sont 2 afd alors $\mathcal{L}(A_1 \times A_2) = \mathcal{L}(A_1) \cap \mathcal{L}(A_2)$ (*)**.

- **Automates finis non déterministes (afnd)**
  - Définis comme un quadruplet (Q, I, F, T) avec Q ensemble des états, I états initiaux, F états finaux, $T \subset Q \times \Sigma \times Q$ ensemble des transitions.
  - Calcul dans un afnd. Calcul réussi.
  - Mots acceptés par un afnd. Langage reconnu.
  - Fonction de transition $\delta : Q \times \Sigma \to \mathfrak{P}(Q)$ et fonction de transition étendue $\delta : Q \times \Sigma^* \to \mathfrak{P}(Q)$ d'un afnd :

    $$\delta(q, c) = \{q' \,/\, (q, c, q') \in T\}.$$

    La notion de blocage se traduit par $\delta(q, c) = \varnothing$.
  - Automate des parties. Algorithme de déterminisation d'un afnd.
  - Les notions suivantes restent valides : états accessibles, co-accessibles, automate émondé, complétion. **L'algorithme de l'automate complétaire ne fonctionne pas sur un automate non déterministe** (déterminiser d'abord).
  - Définition de l'automate produit dans le cas d'un automate non déterministe. L'algorithme de construction pas à pas ne change pas : on fait simplement attention à ce que la lecture d'une lettre depuis un état peut conduire à plusieurs transitions.

- **Automates finis non déterministes à transitions spontanées (ε-afnd)**
  - Définis comme un quadruplet (Q, I, F, T) avec Q ensemble des états, I états initiaux, F états finaux, $T \subset Q \times \Sigma \cup \{\varepsilon\} \times Q$ ensemble des transitions.
  - Les notions de calcul, calcul réussi, mot accepté, langage reconnu, *etc* sont les mêmes que pour un afnd.
  - ε-clôture ou ε-fermeture, notée $\eta : Q \to \mathfrak{P}(Q)$ dans le cours, définie comme l'ensemble des états qu'on peut atteindre à partir d'un état q en n'empruntant que des ε-transitions.

- Formules récursives pour le calcul de la fonction de transition étendue : $\delta : Q \times \Sigma^* \to \mathfrak{P}(Q)$
- Algorithme de suppression des ε-transitions pour passer d'un ε-afnd à un afnd équivalent (sans avoir à déterminiser).
- Algorithme de déterminisation d'un ε-afnd.

- **Langages non reconnaissables** : le lemme de l'étoile et sa démonstration doivent être sus, la méthode pour montrer qu'un langage est non reconnaissable doit être sue.

  - **Lemme de l'étoile (*)** : Soit L est un langage reconnu par un automate à n états et $u \in L$ tel que $|u| \geq n$ alors il existe une décomposition $u = xyz$ telle que $|xz| \leq n$, $y \neq \varepsilon$, $\forall k \in \mathbb{N}, xy^k z \in L$.

  - Application : $L = \{a^n b^n, n \in \mathbb{N}\}$ **n'est pas reconnaissable (*)**

  - Utilisation des propriétés de fermeture des langages reconnaissables pour montrer qu'un langage n'est pas reconnaissable.

NB : le lien entre langages réguliers et langages reconnaissables n'a pas encore été étudié : pas d'automates de Thomson à ce stade